

sat-mb-fidelity___fidelity-chain-f-k4

An AgentMPI run analysis

Abstract

This is the deepest cell of the saturation attempt — seven successive re-summarisations of an accumulator that must eventually carry ninety-six items — and it is the cell in which the campaign’s failure to saturate is most completely explained. Retention is 1.000, all eight ranks intact, no fabricated identifiers, and the reason is visible in the seven merge outputs recovered from the fabric: 490, 387, 505, 634, 531, 613 and 695 tokens for accumulators of 12, 24, 36, 48, 60, 72 and 84 items, against a stated budget of 450. Six of the seven exceeded it, the last by 54%, and every one recorded `attempt: 1`, because the contract stored in this run’s fabric reads `max_tokens: null` with an empty semantics string while `meta` carries `{"max_tokens": 450}`. The budget reached the worker as a generation hint and never reached the checker. Alongside the overrun, the operator re-encoded hard: bytes per identifier in the transmitted accumulator fall 123.6, 68.5, 26.8, 25.9, 21.2, 15.8, 14.6 as the fold proceeds, an $8.5\times$ compression that converges on a packed table (`[F-1-0]107n [F-1-1]108d ...`). An operator permitted both to compress and to overflow is never forced to discard anything, which is why depth 7 scores exactly what depth 2 scores in this campaign, and why the tree-shape hypothesis cannot be tested here at all. The remedy was applied ten minutes of wall-clock later: `inc-mb-fidelity-inc` enforces `max_tokens: 450` on incompressible payloads, and its chain cell — same algorithm, same p , same nominal budget — retains 0.3854 with ranks 4 through 7 erased whole. One knob separates the two.

1 What this run is

property	value
run	sat-mb-fidelity__fidelity-chain-f-k4
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	<code>broker</code> $\times$ 8, <code>worker</code> $\times$ 49
events	118
wall time	1,376.99 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.20×	total busy time over wall time
parallel efficiency	2.5%	achieved parallelism per rank
idle fraction	97.5%	rank-seconds paid for and unused
peak ranks busy	1	of 8 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	2.06×	slowest rank’s busy time over the mean
coordination share	61.9%	rank-seconds blocked in collectives, of those available
coordination in flight	100.0%	wall clock with any rank inside a collective
rank reattachments	49	fresh agent processes that took over a rank role
agent calls	7	invocations that reached a model
agent latency p50 / p95	47.52 / 734.07 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	3,645	0 deferred under rendezvous
tokens in / out	6,795 / 3,855	prompt and completion
cost	\$0.0782	0.0203 \$/kTok out

3 What the analysis notices

- **[critical]** At least one rank waited 908 s for an agent to claim its task before the broker gave up. That is a pool-sizing failure outside the protocol, not a slow run.
- **[warning]** Concurrency never exceeded one rank despite a world size of 8: this ran serially.
- **[note]** Collectives account for 61.9% of the run’s rank-seconds, so more of the pool’s time went to coordination than to work.
- **[warning]** The cost model’s α and β here are a median fallback, not a fit: the slope was negative ($\beta = -2.0594 \text{ s/token}$), meaning latency fell as output grew — so something outside the model dominated the measurement, typically queueing, whose wait enters the latency but not the token count, on a fit with $R^2 = 0.392$. Any latency prediction from this run’s calibration is a median, not a model.
- **[note]** Load imbalance is 2.06×: the slowest rank was busy far longer than the mean.
- **[ok]** 49 rank reattachment(s) occurred, up to incarnation 14: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- **[ok]** All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- **[ok]** All 1 checkable collective(s) also matched the predicted round count.

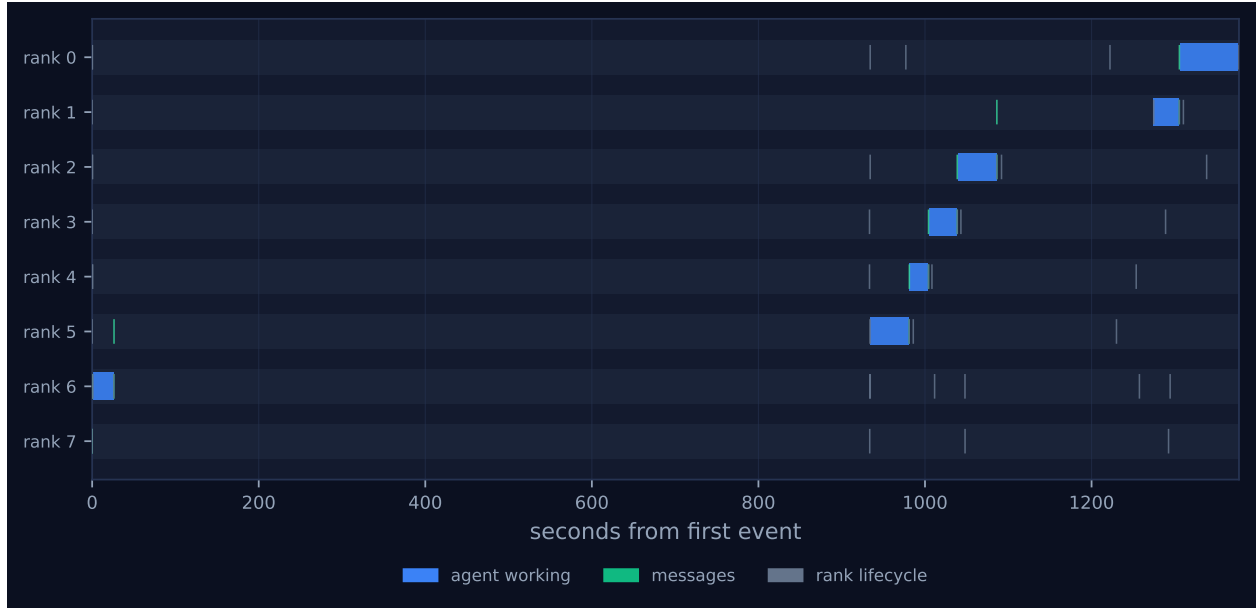


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

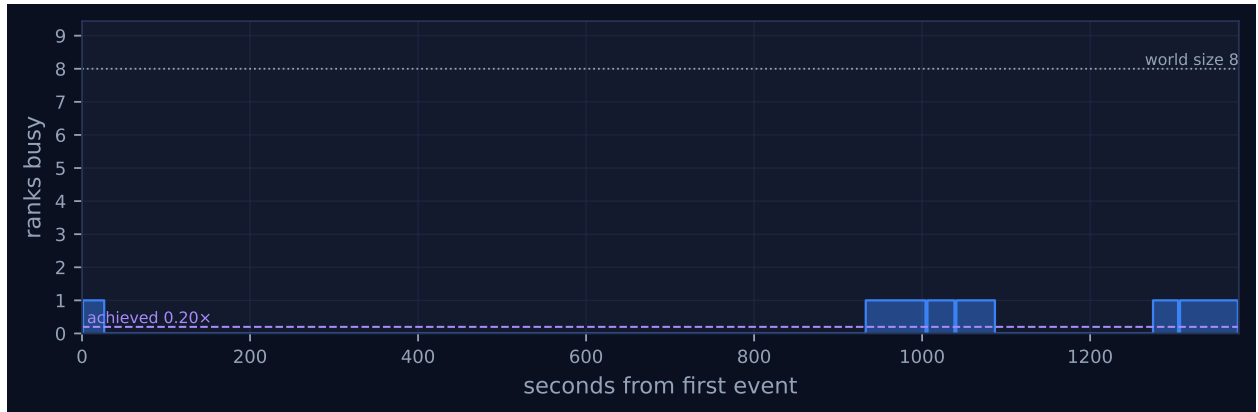


Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

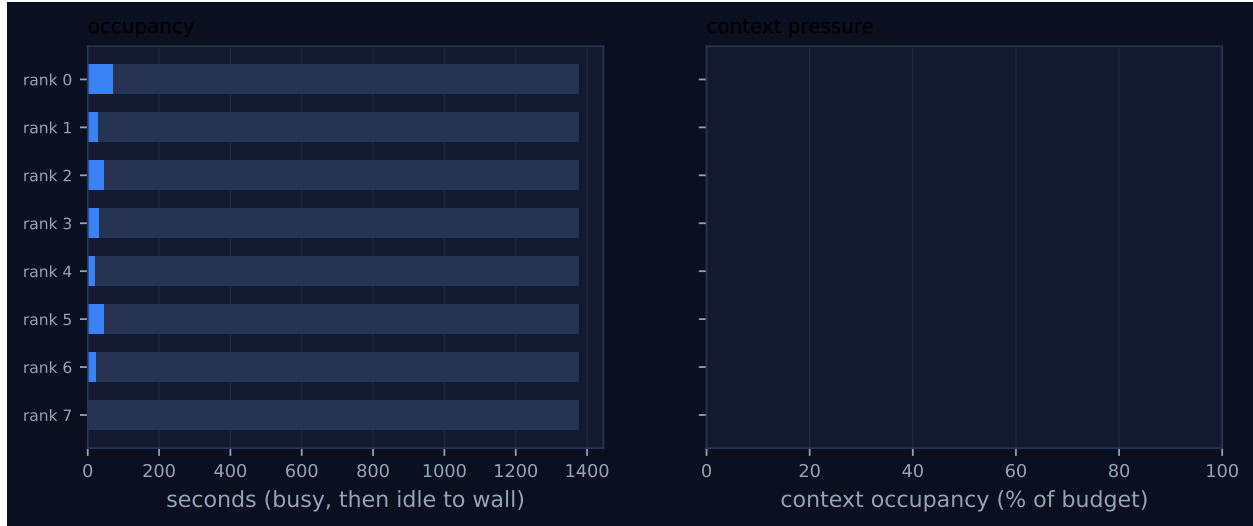


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	70.52	4.2	1	1	0	1	0	0.0	finalized
1	30.27	1.9	1	1	1	1	628	0.0	finalized
2	45.87	2.9	1	1	1	1	545	0.0	finalized
3	32.87	2.1	1	1	1	1	649	0.0	finalized
4	22.47	1.3	1	1	1	1	520	0.0	finalized
5	46.66	3.2	1	1	1	1	402	0.0	finalized
6	25.15	1.6	1	1	1	1	505	0.0	finalized
7	0.00	0.0	0	0	1	0	396	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	chain	—	8	8	7	7	7	7	3,547	1,376.98	1,376.97	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

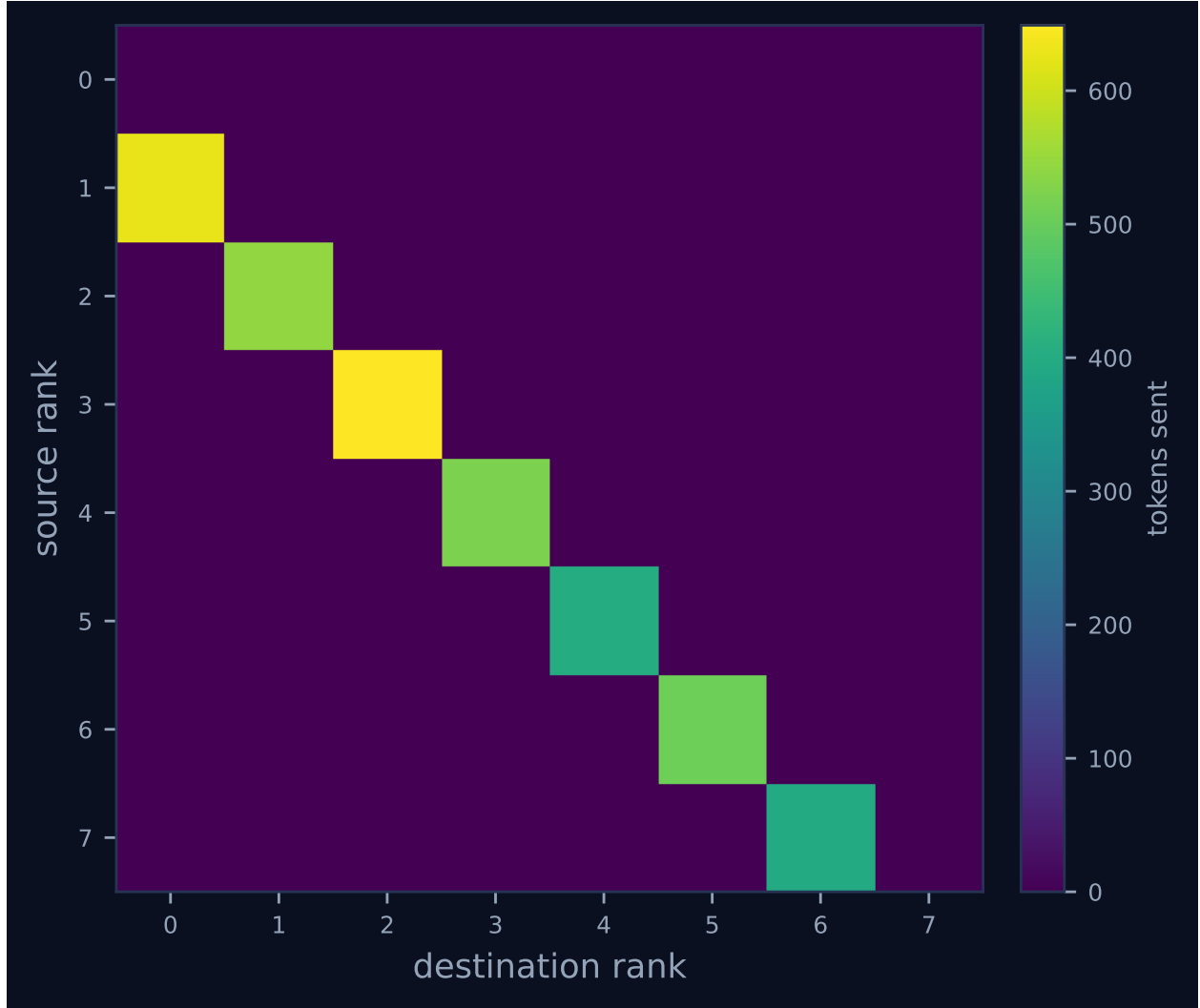


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

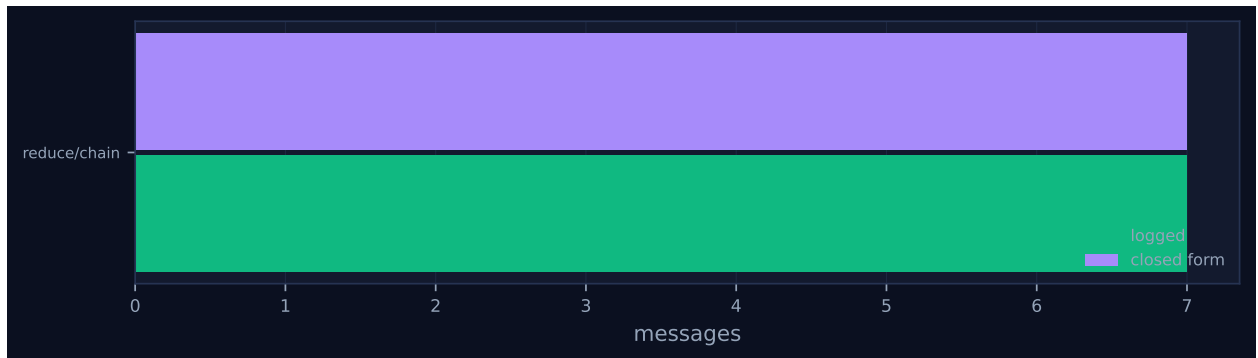


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

6 Reading the timeline

Figure 1 is a staircase running from rank 7 down to rank 0, one bar per lane, which is the chain reduction drawn literally: rank 7 sends its leaf report to rank 6, which merges and sends to rank 5, and so on to the root, so exactly one rank is ever busy and the critical path is the whole run. Peak ranks busy is 1 of 8 and the serial fraction of active time is 100%. Two features of the picture are not the algorithm, however, and they dominate it. The first is the enormous gap between rank 6’s bar, which ends at $t = 26$ s, and rank 5’s, which does not begin until $t = 934$ s. Differencing every `broker.enqueue` $\rightarrow$ `broker.claim` $\rightarrow$ `broker.complete` triple in the log localises it precisely: rank 5’s `merge:d2` sat enqueued for 907.71 s before any worker claimed it, and rank 1’s `merge:d6` for a further 188.35 s. Summing the seven waits gives 1,100.32 s of the 1,376.99 s to `job.finish` — 79.9% of the run — against 273.82 s of claim-to-complete generation, which matches `busy_union_s` of 273.102 to within the event granularity. The second is the 49 reattachments and the 57 `rank.init` events for eight ranks: the pool churned throughout, and the log continues to 1,682.14 s past `job.finish`, which is why the dashboard’s rank health table shows every rank red and suspected `fail_stop` while `metrics.json` records `ok: true`, zero failures and zero contract violations.

This makes the run’s headline wall time unusable as an algorithmic measurement, and it is worth saying plainly because the four cells of this campaign are otherwise directly comparable: 1,376.98 s here against the flat fold’s 197.62 s, the binomial tree’s 77.56 s and the k -ary tree’s 53.07 s. The structural prediction for chain against binomial at $p = 8$ is $7/3 = 2.33\times$; the measurement is $17.8\times$. Take the queue wait out and the seven merges cost 273.8 s, giving $3.5\times$, which is the figure I would defend. It is also the origin of the flagged calibration warning: the regression returned $\beta = -2.0594$ s per token on $R^2 = 0.392$, latency apparently falling as output grew, because the two longest-latency calls are the two that waited in the queue and the wait enters the latency without entering the token count. The guard was right to reject the fit, and `alpha_p50` of 47.5 s is a median, not a model.

One metric does not survive contact with the log. `max_claim_wait_s` reads 0.0 for a run containing a 907-second enqueue-to-claim interval, because the metric is computed from `broker.expire` events — waits that ended in a timeout — rather than from the enqueue-to-claim difference for calls that were eventually claimed. Every call here was claimed and completed, so nothing expired, and the single most consequential quantity in the run is reported as zero. I would report this as a real gap in the instrumentation rather than a property of the run: `coordination_share` of 61.9% is the only metric that notices anything, and it notices it for the wrong reason, since it counts the downstream ranks’ blocked receives rather than the broker starvation that caused them.

7 Interpretation

What the campaign varied is legible from the config and it is not what the `sat-` prefix suggests was achieved. Against `real-mb-fidelity` — same p , same broker executor, same three algorithms plus `kary` — this campaign raised `facts` from 4 to 12 and lowered `merge_budget` from 900 to 450. The root must therefore render 96 items into half the space the baseline had for 32. On the baseline’s own encoding, roughly 19 tokens per item, 450 tokens holds about 23 items, so the fold should have saturated after two ranks and stayed saturated for five more. It did not saturate at all, in any of the four cells, and this one shows both escape routes in a single monotone sequence. The operator re-encodes: rank 7’s leaf report spends 1,483 bytes on 12 items and the accumulator arriving at the root spends 1,225 bytes on 84, with the intermediate forms visibly abandoning prose for a packed table — `[F-3-0] 121 nom` at depth 4, then `[F-2-0] 114n [F-2-1] 115d ...` at depth 5, where

the `findings` array collapses from 60 strings to 6 because a dozen items are packed into each. And when re-encoding runs out, the operator simply exceeds the number: 490 tokens at the first merge, already 9% over, and 695 at the last, 54% over. The prompt itself licenses the first route (“You may compress wording, but you may not drop an item”) and the run’s plumbing licenses the second. Saturation was never reached, so this campaign contributes a negative result about the design rather than a measurement of retention under pressure, and that negative result is what motivated the two changes visible in the repository at `HEAD: merge_contract(budget)`, which puts `max_tokens` in the contract with a semantics clause telling the operator to omit whole items rather than re-encode, and `-incompressible`, which removes the re-encoding route by making the payload high-entropy.

The structural numbers are true by construction and both model checks pass: seven messages against $p - 1 = 7$, seven rounds against the closed form, `fold_depth: 7`. This cell therefore establishes the reference point at the deep end of the campaign’s depth range, which spans 2 (`kary`), 3 (`binomial`) and 7 (`chain` and `flat`) — three distinct depths across four algorithms, because `flat` is not a one-shot p -way fold: `algorithms.py` applies the binary kernel $p - 1$ times at the root, so it too has `fold_depth: 7`, with `rounds: 1`. That all four score retention 1.000 with identical per-rank vectors is the campaign’s whole result, and under an unenforced budget it is uninformative about the depth question rather than evidence against it. The informative version of the same comparison is in `inc-mb-fidelity-inc`, where all four cells again agree exactly, this time at 0.3854 with the same 37 survivors; there the agreement does refute the hypothesis, because the budget is binding.

Of the flagged findings, the serial-execution warning and the $2.06\times$ imbalance are the algorithm’s definition rather than defects — 2.06 is in fact the mildest imbalance in the campaign, which is an artefact of the queue wait inflating the mean rather than a virtue of the chain. The 49 reattachments are the broker pool’s normal mode. The calibration warning is a genuine warning about this run’s numbers, correctly raised. The model-check finding is the check passing. What is not flagged, and should be, is the queue starvation.

8 Threats to this reading

The executor is real — `broker` $\times$ 8 and `worker` $\times$ 49 — so this is model behaviour, and it must not be pooled with the `function-executor` cells in `fid-synth` and `fid-inc-smoke`, whose operator drops whole accumulator blobs at a fixed probability and therefore implements a loss model rather than exhibiting one.

The most serious threat is that retention 1.000 does not measure information transport. `make_fact_report` builds item i of rank r with throughput $100 + 7r + i$ under workload `['nominal', 'degraded', 'saturated'] [i % 3]`, so the payload is a deterministic function of the identifier, and the packed encoding this run converges on reproduces the rule literally: `[F-1-0]107n [F-1-1]108d [F-1-2]109s` is 107, 108, 109 with the workload initial. A rank whose message had been lost entirely could be reconstructed by an operator that had merely noticed the pattern, and it would score as retained. The chain is the cell where this bites hardest, because its accumulator is a strictly increasing rank prefix in an arithmetic sequence and each merge sees the next rank only as the continuation of a series it has already seen six times. This is precisely why `-incompressible` exists, and why the value-level metrics (`n_intact`, `n_label_only`, `value_retention`) are reported only for the `-inc` campaigns: here I can verify that 96 identifiers arrived, not that 96 facts did.

Second, single trial, and one whose timing is contaminated as described. Per-call generation latencies were 25.15, 46.67, 22.47, 32.87, 45.87, 30.27 and 70.52s on prompts of 830–1,077 tokens, a $3.1\times$

spread with no relation to prompt size, so any per-merge cost I quote is a single draw from a wide distribution. The $8.5\times$ compression figure is likewise one operator’s choices on one pass; the sibling cells’ depth-1 merges at the same prompt structure returned between 211 and 491 tokens, so per-item cost in the compressible regime is a decision made afresh at each application and not a constant of the configuration.

Third, the four cells of this campaign ran sequentially against one persistent pool of Cursor subagents, so cross-run text reuse was mechanically possible, and two of this campaign’s late accumulators are byte-identical across cells. I ran the determinism test the contract requires: prompt digest `edf28c1217be` appears in both this campaign’s `flat` and `binomial` cells and returns different results (354 against 211 tokens, different digests), so identical input does not give identical output here and determinism cannot explain the identities. Convergence on the canonical packed encoding can, and is consistent with the identities occurring only among the dense late accumulators, but pool memory cannot be excluded from the trace.

Finally the accounting caveats. Every `msg.recv` records `admitted: false` because the collective’s internal receives pass `admit=False`, so context occupancy reads 0% on ranks that consumed up to 1,077 prompt tokens and this run says nothing about admission pressure. The \$0.0782 is modelled from \$3/Mtok in and \$15/Mtok out with `tokenizer_exact: false`, not billed, and the same approximate counter produced the 490/695-token measurements, so “54% over budget” is internally consistent rather than externally exact. And `rank_position_correlation: 0.000` is not a fairness result but an undefined quantity: at retention 1.000 the per-rank variance is zero and the code returns 0.0 through a guard.